Homework 6 - Fiacchi, Szczurek, Santini

Giada Fiacchi, Lucia Santini, Aleksandra Szczurek

2025-11-27

EXERCISE 1

Write an algorithm to generate a sample of $n = 1000$ random variates from the mixture of three Normal distributions:

$$f(x) = \sum_{i=1}^{3} \pi_k f_k(x)$$

where $f_k \sim N(\mu_k, \sigma_k^2)$, with $\mu_k = (-1, 1, 3)$, $\sigma_k^2 = (0.25, 0.25, 2.25)$ and $\pi_k = (0.25, 0.25, 0.5)$, $k = 1, \ldots, 3$.

Graph the histogram of the sample with the theoretical mixture density superimposed. Is the resulting mixture density unimodal or multimodal shaped? Find a different configuration of parameters μ_k and σ_k^2 so that the resulting density exhibits a asymmetric unimodal shape.

```r
#define the parameters and f(x)
mu <- c(-1, 1, 3)
var <- c(0.25, 0.25, 2.25)
sigma <- sqrt(var)
pi <- c(0.25, 0.25, 0.5)

fmix <- function(x) pi[1]*dnorm(x, mu[1], sigma[1])+
  pi[2]*dnorm(x, mu[2], sigma[2])+
  pi[3]*dnorm(x, mu[3], sigma[3])

#Generate 1000 values from each of the three Normal distributions.
x1 <- rnorm(1000, mu[1], sigma[1])
x2 <- rnorm(1000, mu[2], sigma[2])
x3 <- rnorm(1000, mu[3], sigma[3])

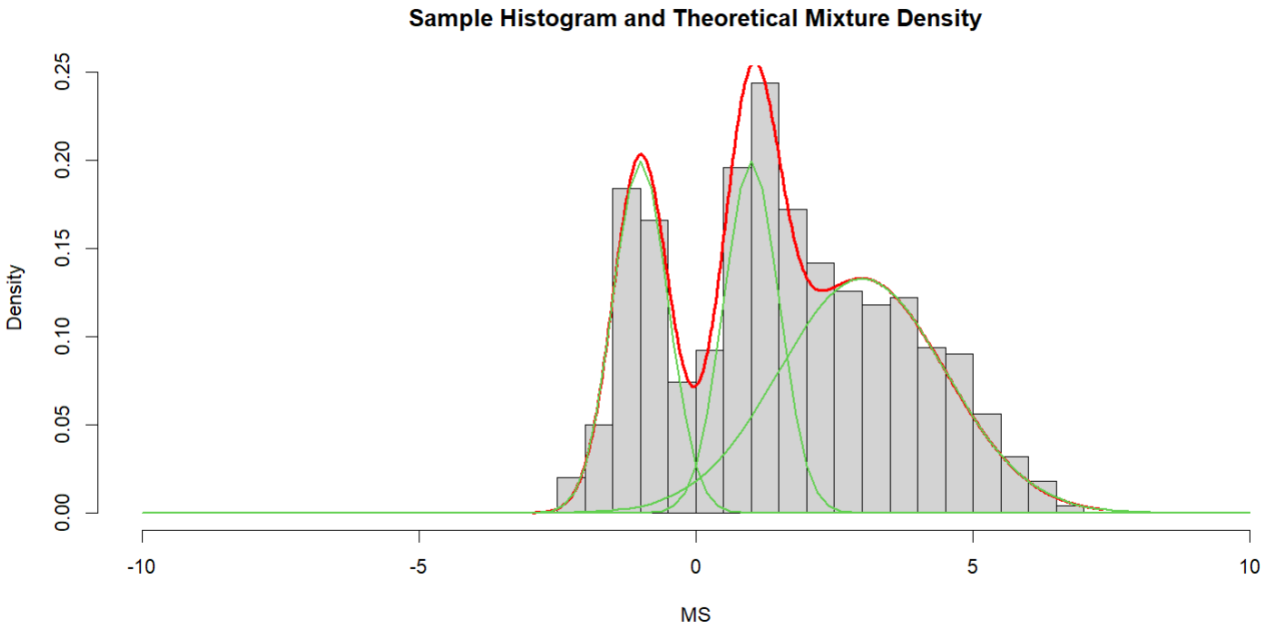
component_choice <- sample(x = 1:3, size = 1000,
  replace = TRUE, prob = pi) # we assign 1, 2 or 3 based on the probability
weights pi.

#We create the final mixture sample (MS) by conditional assignment.
# Start by initializing X with the first component pool
MS <- x1
# Replace elements where component_choice was 2 with the value from x2
MS[component_choice == 2] <- x2[component_choice == 2]

# Replace elements where component_choice was 3 with the value from X3
```

```r
MS[component_choice == 3] <- x3[component_choice == 3]

#histogram
hist(MS, xlim=c(-10, 10), freq = FALSE, breaks = 30, main = "Sample Histogram and
Theoretical Mixture Density", ylim=c(0, 0.25))
# Superimpose the theoretical density curve
x <- seq(min(MS) - 0.5, max(MS) + 0.5, length.out = 500)
theoretical_density <- fmix(x)
lines(x, theoretical_density, col = "red", lwd = 2.2)
curve(pi[1]*dnorm(x,mu[1],sigma[1]),add=TRUE,col=3, lwd=1.5)
curve(pi[2]*dnorm(x,mu[2],sigma[2]),add=TRUE,col=3, lwd=1.5)
curve(pi[3]*dnorm(x,mu[3],sigma[3]),add=TRUE,col=3, lwd=1.5)
```



The resulting mixture density is multimodal (three distinct peaks), due to the large separation between the means relative to the small standard deviations. To achieve an asymmetric unimodal shape, you need to increase the overlap (by increasing variance and/or reducing the distance between means) while keeping the weights/variances skewed.

A suitable alternative configuration is: mu(0, 1, 2), var: (1.5, 1.0, 1.5), pi:(0.2, 0.4, 0.4)

```r
#We try with these new parameters:

mu2 <- c(0, 1, 2)
var2 <- c(1.5, 1, 0.5)
sigma2 <- sqrt(var2)
pi2 <- c(0.4, 0.4, 0.2)

fmix2 <- function(x) pi2[1]*dnorm(x, mu2[1], sigma2[1])+
  pi2[2]*dnorm(x, mu2[2], sigma2[2])+
  pi2[3]*dnorm(x, mu2[3], sigma2[3])
```

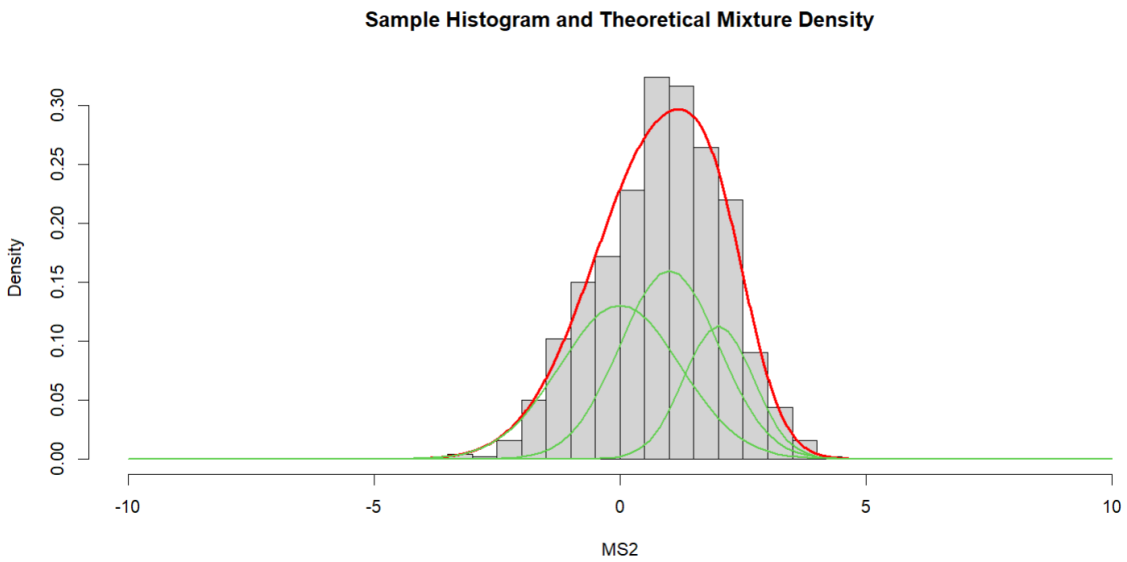
```r
#Generate 1000 values from each of the three Normal distributions.
x1_2 <- rnorm(1000, mu2[1], sigma2[1])
x2_2 <- rnorm(1000, mu2[2], sigma2[2])
x3_2 <- rnorm(1000, mu2[3], sigma2[3])


component_choice2 <- sample(x = 1:3, size = 1000,
                            replace = TRUE, prob = pi2
) # we assign 1, 2, or 3 based on the probability weights pi.

#We create the final mixture sample (MS) by conditional assignment.
# Start by initializing X with the first component pool
MS2 <- x1_2
# Replace elements where component_choice was 2 with the value from x2
MS2[component_choice2 == 2] <- x2_2[component_choice2 == 2]

# Replace elements where component_choice was 3 with the value from X3
MS2[component_choice2 == 3] <- x3_2[component_choice2 == 3]

#histogram
hist(MS2, xlim=c(-10, 10), freq = FALSE, breaks = 25, main = "Sample Histogram and
Theoretical Mixture Density", ylim=c(0, 0.35))
# Superimpose the theoretical density curve
x_2 <- seq(min(MS2) - 0.5, max(MS2) + 0.5, length.out = 500)
theoretical_density2 <- fmix2(x_2)
lines(x_2, theoretical_density2, col = "red", lwd = 2.2)
curve(pi2[1]*dnorm(x,mu2[1],sigma2[1]),add=TRUE,col=3, lwd=1.5)
curve(pi2[2]*dnorm(x,mu2[2],sigma2[2]),add=TRUE,col=3, lwd=1.5)
curve(pi2[3]*dnorm(x,mu2[3],sigma2[3]),add=TRUE,col=3, lwd=1.5)
```



We obtained an asymmetric unimodal shape.

EXERCISE 2

Consider the following optimization problems:

(a) maximise f(x, y) = 5 − x^2 − y^2

(b) minimise f(x, y) = x^2 + 2*y^2 - xy + 14y

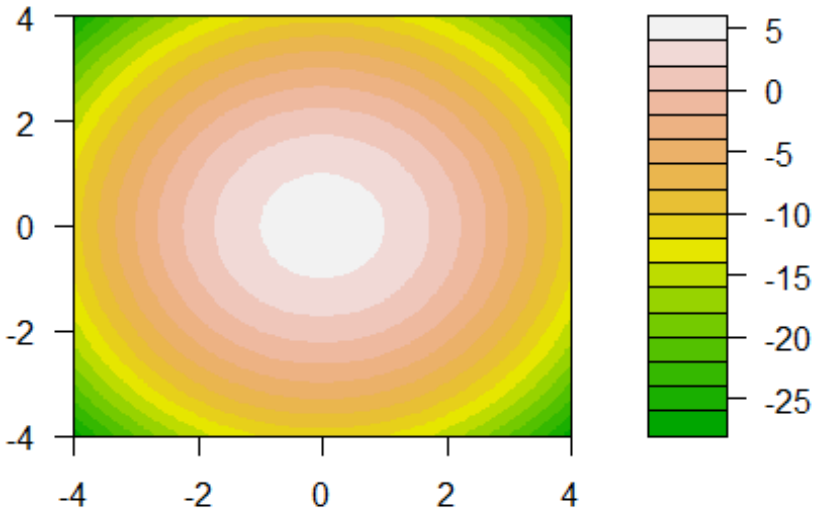
(c) minimise f(x, y) = x/(x^2 + y^2 + 4)

Draw the contour-plot of the above functions and the 3D graph (use function GA::persp3D, package GA) to visually explore the domain of the functions. Use both Newton-Raphson and Nelder-Mead algorithm to find maximum/minimum of the above functions and compare the results. For both algorithm, try different starting values to monitor the convergence to the global maximum. For different starting points, plot intermediate values of the Newton-Raphson sequence obtained at each iteration to see how the algorithm reaches the mode.

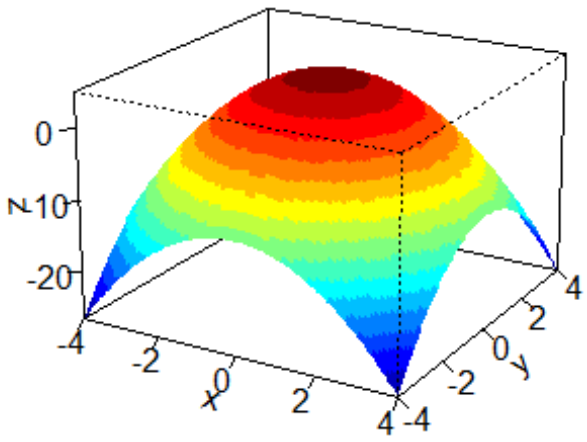
```r
#Install the package GA
library(GA)

#First function: f(x, y) = 5 - x^2 - y^2
f <- function (x,y) 5 - x^2 - y^2
ngrid <- 100
x <- seq(-4,4,length=ngrid)
y <- x
z <- outer(x,y,f)

#contour plot
filled.contour(x,y,z, color.palette=terrain.colors)
```




```
#3D graph
GA::persp3D(x,y,z, border=NA)
```




```
#define the function
fa <- function(par) -(5 - par[1]^2 - par[2]^2)
```

```r
#Newton-Raphson
#(0,0)
NRf1 <- nlm(fa, p=c(0,0))
NRf1$minimum
```

```
## [1] -5
```

```r
#Nelder Method
#(0,0)
NMf1 <- optim(fa, p=c(0,0), method="Nelder-Mead")
NMf1$value
```

```
## [1] -5
```

```r
#table for comparison
tab<- data.frame(
  Method = c("Newton Raphson", "Nelder Method"),
  Value = c(NRf1$minimum, NMf1$value)
)
tab
```

```
##           Method Value
## 1 Newton Raphson    -5
## 2  Nelder Method    -5
```

```r
#Newton-Raphson with different starting values
#(3,0)
NRf2 <- nlm(fa, p=c(3,0))
NRf2$minimum
```

```
## [1] -5
```

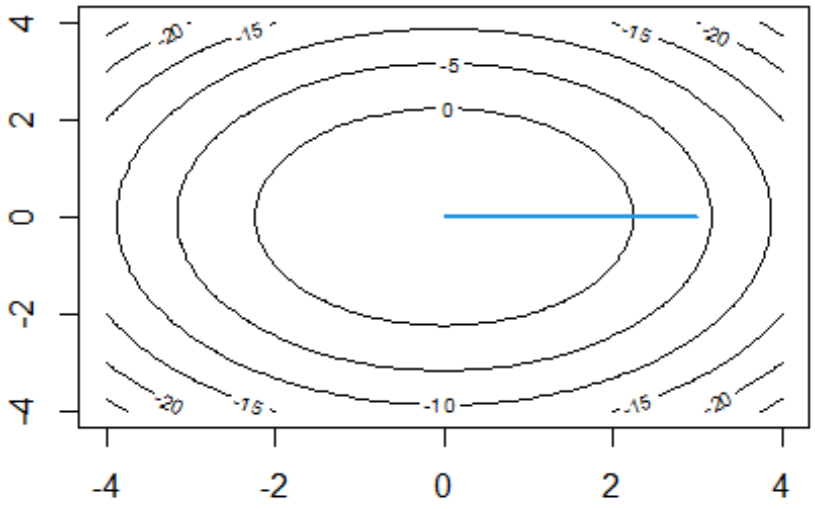
```r
#(-2,2)
NRf3 <- nlm(fa, p=c(-2,2))
NRf3$minimum
```

```
## [1] -5
```

```r
# plot intermediate values of the Newton-Raphson sequence obtained at each
iteration
#(3,0)
sta <- c(3,0)
it <- nlm(fa, p=sta)$iterations
path <- sta

for(i in 1:it){
  est <- nlm(fa, p=sta, iterlim=i)$estimate
  path <- rbind(path, est)
}

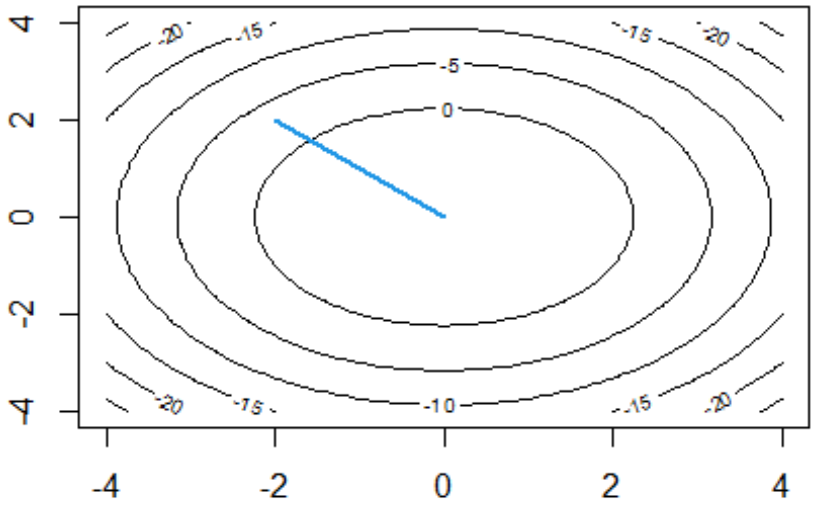
contour(x,y,z)
lines(path, col=4, lwd=2, pch=19)
```




```r
#(-2,2)
sta <- c(-2,2)
it <- nlm(fa, p=sta)$iterations
path <- sta

for(i in 1:it){
  est <- nlm(fa, p=sta, iterlim=i)$estimate
  path <- rbind(path, est)
}

contour(x,y,z)
lines(path, col=4, lwd=2, pch=19)
```




```
#Nelder-Method with different starting values
#(3,0)
NMf2 <- optim(fa, p=c(3,0), method="Nelder-Mead")
NMf2$value

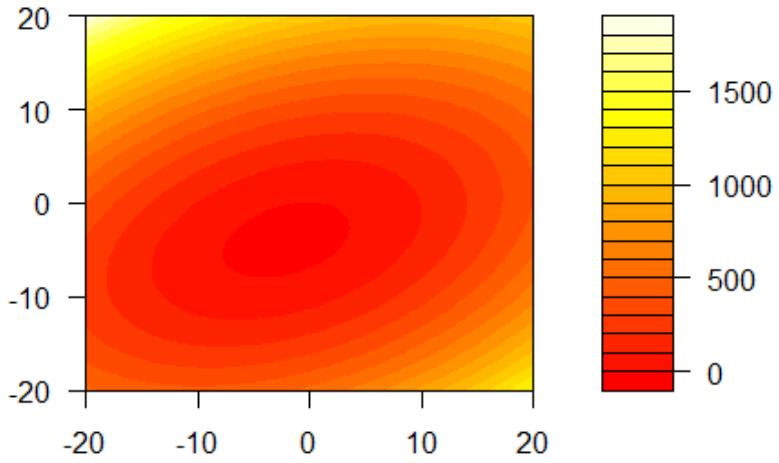
## [1] -5

#(-2,2)
NMf3 <- optim(fa, p=c(-2,2), method="Nelder-Mead")
NMf3$value

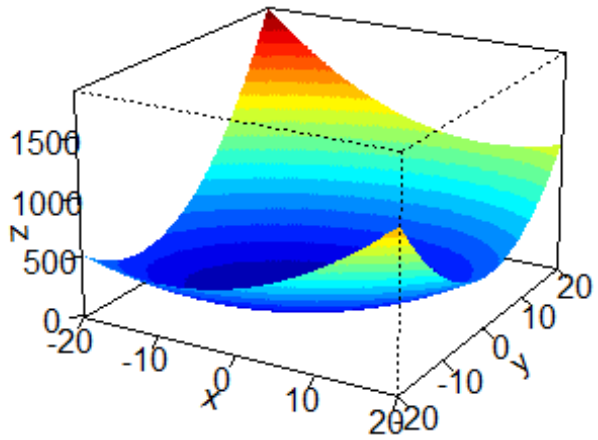
## [1] -5

#Second function: f(x, y) = x^2 + 2*y^2 - xy + 14y
g <- function (x,y) x^2 + 2*y^2 - x*y + 14*y
ngrid <- 200
x <- seq(-20,20,length=ngrid)
y <- seq(-20,20,length=ngrid)
z <- outer(x,y,g)

#contour plot
filled.contour(x,y,z, color.palette=heat.colors)
```




```
#3D graph
GA::persp3D(x,y,z, border=NA)
```




```
#define the function
gb <- function(par) ( par[1]^2 + 2*par[2]^2 - par[1]*par[2] + 14*par[2])

#Newton-Raphson
#(0,0)
NRg1 <- nlm(gb, p=c(0,0))
NRg1$minimum
```

```
## [1] -28
```

```
#Nelder-method
#(0,0)
NMg1 <- optim(gb, p=c(0,0), method="Nelder-Mead")
NMg1$value
```

```
## [1] -28
```

```
#table for comparison
tab<- data.frame(
  Method = c("Newton Raphson", "Nelder Method"),
  Values = c(NRg1$minimum, NMg1$value)
)
tab
```

```
##           Method Values
## 1 Newton Raphson    -28
## 2  Nelder Method    -28
```

```
#Newton-Raphson with different starting values
#(-10,10)
NRg2 <- nlm(gb, p=c(-10,10))
NRg2$minimum
```

```
## [1] -28
```

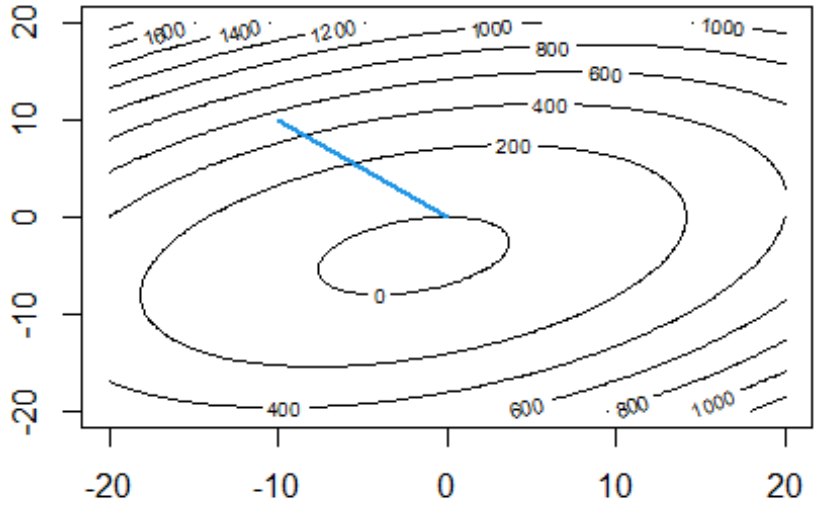
```
#(5,-10)
NRg3 <- nlm(gb, p=c(5,-10))
NRg3$minimum
```

```
## [1] -28
```

```
# plot intermediate values of the Newton-Raphson sequence obtained at each
iteration
#(-10,10)
sta <- c(-10,10)
it <- nlm(fa, p=sta)$iterations
path <- sta

for(i in 1:it){
  est <- nlm(fa, p=sta, iterlim=i)$estimate
  path <- rbind(path, est)
}

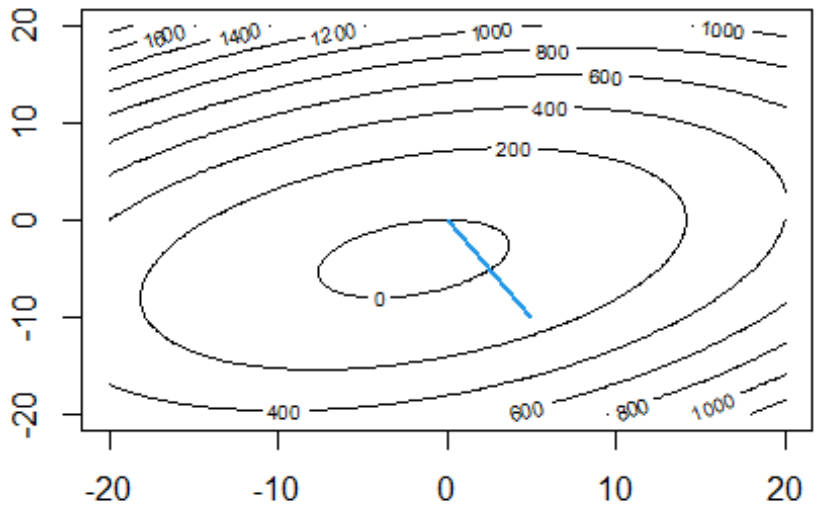
contour(x,y,z)
lines(path, col=4, lwd=2, pch=19)
```




```
#(5,-10)
sta <- c(5,-10)
it <- nlm(fa, p=sta)$iterations
path <- sta

for(i in 1:it){
  est <- nlm(fa, p=sta, iterlim=i)$estimate
  path <- rbind(path, est)
}

contour(x,y,z)
lines(path, col=4, lwd=2, pch=19)
```




```
#Nelder-Method with different starting values
#(-10,10)
NMg2 <- optim(gb, p=c(-10,10), method="Nelder-Mead")
NMg2$value

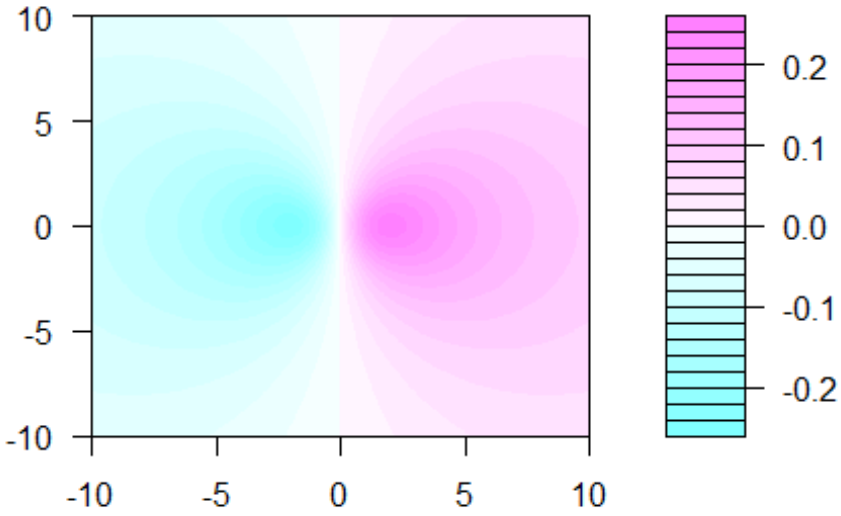
## [1] -28

#(5,-10)
NMg3 <- optim(gb, p=c(5,-10), method="Nelder-Mead")
NMg3$value

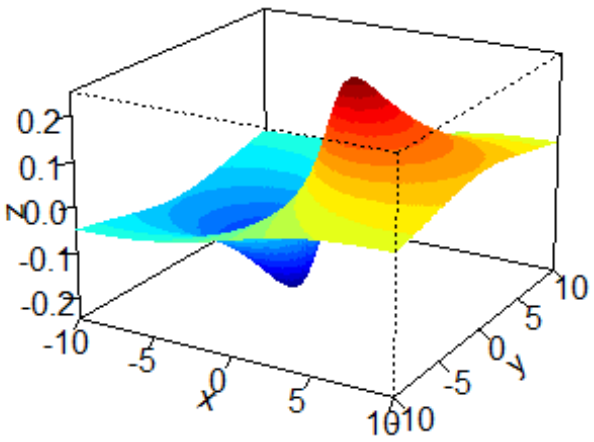
## [1] -28

#Third function: f(x, y) = x/(x^2 + y^2 + 4)
h <- function (x,y) x/(x^2 + y^2 + 4)
ngrid <- 100
x <- seq(-10,10,length=200)
y <- seq(-10,10,length=200)
z <- outer(x,y,h)

#contour plot
filled.contour(x,y,z, color.palette=cm.colors)
```




```
#3D graph
GA::persp3D(x,y,z, border=NA)
```




```
#define the function
hc <- function(par) ( par[1]/ (par[1]^2 + par[2]^2 + 4))
```

```r
#Newton-Raphson
#(0,0)
NRh1 <- nlm(hc, p=c(0,0))
NRh1$minimum
```

```
## [1] -0.25
```

```r
#Nelder-Method
#(0,0)
NMh1 <- optim(hc, p=c(0,0), method="Nelder-Mead")
NMh1$value
```

```
## [1] -0.25
```

```r
#table for comparison
tab<- data.frame(
  Method = c("Newton Raphson", "Nelder Method"),
  Values = c(NRh1$minimum, NMh1$value)
)
tab
```

```
##           Method Values
## 1 Newton Raphson  -0.25
## 2  Nelder Method  -0.25
```

```r
#Newton-Raphson with different starting values
#(0,2)
NRh2 <- nlm(hc, p=c(0,2))
NRh2$minimum
```

```
## [1] -0.25
```

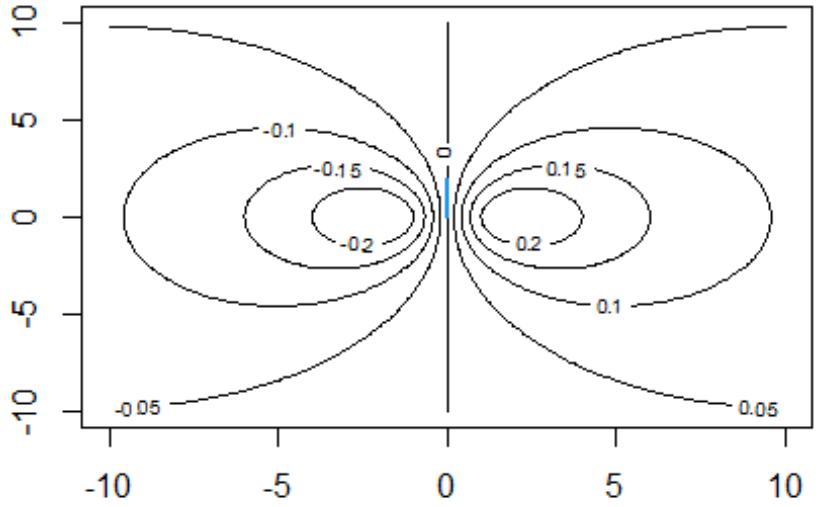
```r
#(-3,3)
NRh3 <- nlm(hc, p=c(-3,3))
NRh3$minimum
```

```
## [1] -0.25
```

```r
# plot intermediate values of the Newton-Raphson sequence obtained at each
iteration
#(0,2)
sta <- c(0,2)
it <- nlm(fa, p=sta)$iterations
path <- sta

for(i in 1:it){
  est <- nlm(fa, p=sta, iterlim=i)$estimate
  path <- rbind(path, est)
}

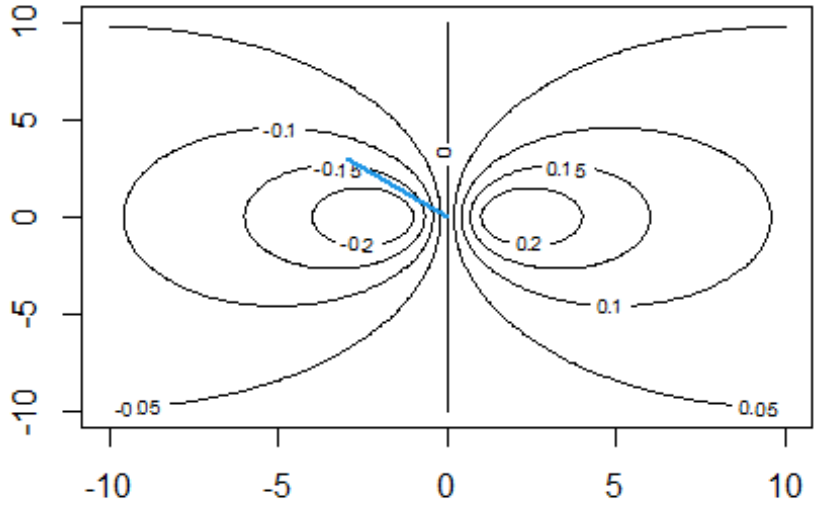
contour(x,y,z)
lines(path, col=4, lwd=2, pch=19)
```




```
#(-3,3)
sta <- c(-3,3)
it <- nlm(fa, p=sta)$iterations
path <- sta

for(i in 1:it){
  est <- nlm(fa, p=sta, iterlim=i)$estimate
  path <- rbind(path, est)
}

contour(x,y,z)
lines(path, col=4, lwd=2, pch=19)
```




```
#Nelder-Method with different starting values
#(0,2)
NMh2 <- optim(hc, p=c(0,2), method="Nelder-Mead")
NMh2$value

## [1] -0.25

#(-3,3)
NMh3 <- optim(hc, p=c(-3,3), method="Nelder-Mead")
NMh3$value

## [1] -0.25
```

EXERCISE 3

Consider the dataset wine, which can be downloaded from UCI Machine Learning Repository (https://archive.ics.uci.edu/ml/datasets/wine). The main task of data analysis is clustering, i.e. using chemical analysis to distinguish the origin of wines. The dataset has 178 observations and 14 variables, in which the origin of wines (Class) is a categorical variable (with three categories that identify the true components) and the others are continuous variables. For this application consider only r = 5 response variables: Alcohol, Alcalinity of ash, Total phenols,Flavanoids, Color intensity. Organize these variables in a new dataset named wine.new. Variable Class will be used for the true classification.

```
library(data.table)

library(mclust)

#download the dataset
wine = fread("https://archive.ics.uci.edu/ml/machine-learning-
databases/wine/wine.data", header = F)
names(wine) = c("Class", "Alcohol", "Malic acid", "Ash", "Alcalinity of ash",
"Magnesium",
                "Total phenols", "Flavanoids", "Nonflavanoid phenols",
"Proanthocyanins",
                "Color intensity", "Hue", "OD280/OD315 of diluted wines",
"Proline")

#response variables
response_vars = c("Alcohol", "Alcalinity of ash", "Total phenols", "Flavanoids",
"Color intensity")

#dataset with classes
wine.new <- wine[, c("Class", response_vars), with = FALSE]

#dataset with true classes
wine.new.1 <- wine[, ..response_vars] #without classes
```

(a) Analyze the wine.new dataset with appropriate descriptive statistics (summary, boxplot for each class, scatter plot, and so on).

```
#summary statistics
summary(wine.new)

##      Class          Alcohol      Alcalinity of ash Total phenols
##  Min.   :1.000   Min.   :11.03   Min.   :10.60     Min.   :0.980
##  1st Qu.:1.000   1st Qu.:12.36   1st Qu.:17.20     1st Qu.:1.742
##  Median :2.000   Median :13.05   Median :19.50     Median :2.355
##  Mean   :1.938   Mean   :13.00   Mean   :19.49     Mean   :2.295
##  3rd Qu.:3.000   3rd Qu.:13.68   3rd Qu.:21.50     3rd Qu.:2.800
##  Max.   :3.000   Max.   :14.83   Max.   :30.00     Max.   :3.880
```

```
##    Flavanoids    Color intensity
##  Min.   :0.340   Min.   : 1.280
##  1st Qu.:1.205   1st Qu.: 3.220
##  Median :2.135   Median : 4.690
##  Mean   :2.029   Mean   : 5.058
##  3rd Qu.:2.875   3rd Qu.: 6.200
##  Max.   :5.080   Max.   :13.000
```

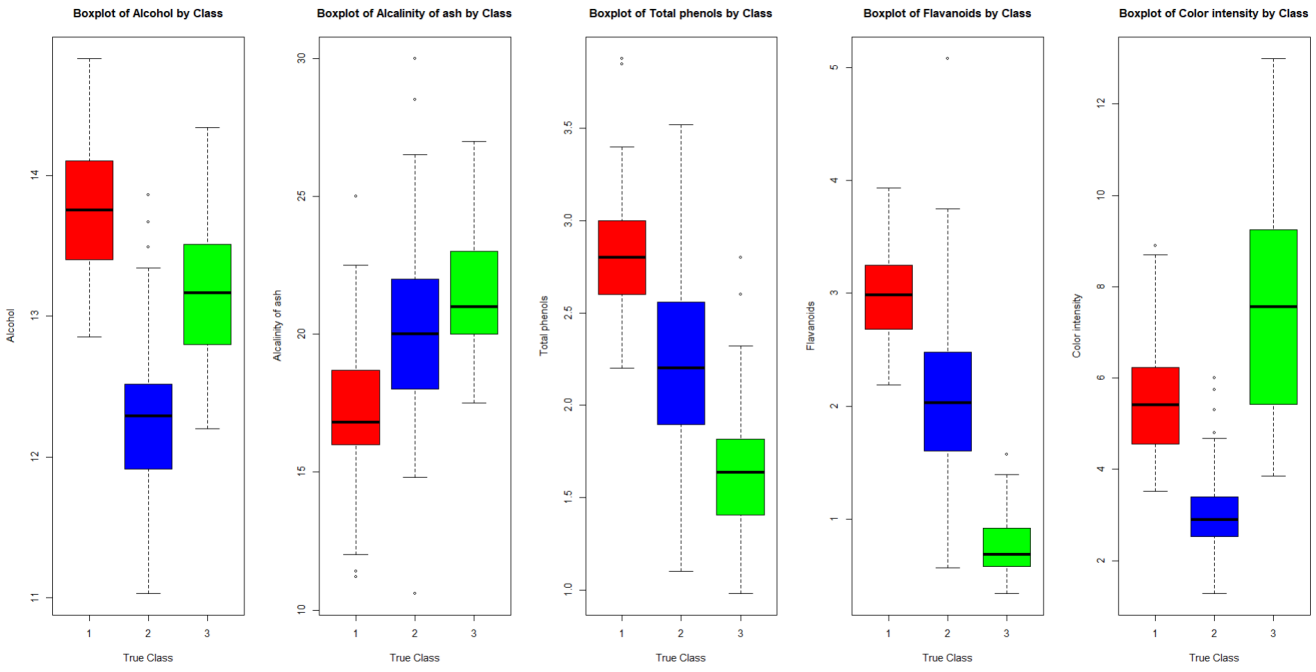
To visually analyze these results, we can create box plots and scatter plots. We are interested in noting which variable best succeeds in separating the three types of classes.

```r
colors = c("red", "blue", "green")

#first class = red
#second class = blue
#third class = green

par(mfrow = c(1,5))

for (var in response_vars) {
  boxplot(wine.new[[var]] ~ wine.new$Class,
          main = paste("Boxplot of", var, "by Class"),
          xlab = "True Class", ylab = var,
          col = colors)
}
```



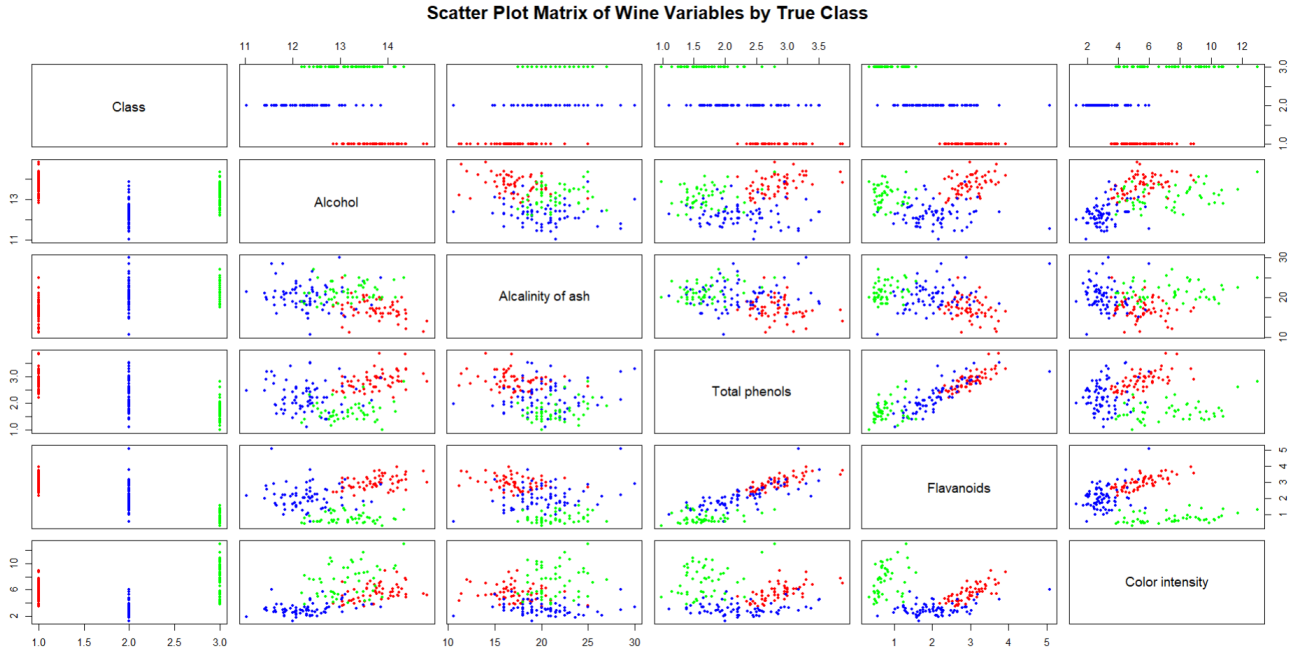
From the visual analysis, it emerges that the variable with the best discriminatory power is Flavanoids. The three generated box plots are well separated, and their medians never overlap, indicating excellent discriminatory power for separating all three classes.

The second variable with good discriminatory power is Total phenols, but the second class slightly overlaps with the first.

The box plots of all the other variables overlap to a greater extent, indicating a lower discriminatory power.

Even from this visual analysis, it can be inferred that the main variables we will use for clustering are those with the highest discriminatory power.

```
#scatter plots
pairs(wine.new,
      col = colors[wine.new$Class], # Applica i colori fissi in base al livello di
class_true
      pch = 20,
      main = "Scatter Plot Matrix of Wine Variables by True Class")
```



From these graphs, we can analyze the pairwise relationships between the five variables and how the possible combinations influence the separation of the three clusters. Some of the combinations show efficiency in separating the three types of classes, especially the pair Flavanoids - Total phenols and the pair Flavanoids - Color intensity.

The visual analysis is important for understanding the optimal structure of the Gaussian Mixture Model (GMM). As we can note from the shape of the point clouds that have formed, none of the shapes are spherical; instead, they all appear elongated and elliptical. From this detail, we can deduce that the GMM model will use flexible covariance (or full covariance) to capture the shape and orientation of the clusters.

Mixture of Multivariate Gaussian models are able to model groups of observations in ellipsoidal regions of the Euclidean space with any orientation and size. Therefore, this will be very useful for performing the analysis. Furthermore, the elongated shapes of the points indicate a strong

correlation between the pairs of variables considered. Finally, if the clusters are visually distinct in a plot, a Gaussian Mixture Model is likely to perform well

(b) Apply the function Mclust to wine.new data in order the estimate model parameters of a mixture of Gaussian distributions. Use the default input arguments to select the best model in terms of number of mixture components and structure of the covariance matrix (do not use any constraint). Analyze the estimation results, in terms of model selection, parameter estimates, clustering and uncertainty.

```
#The Mclust function estimates parameters of a Gaussian finite mixture model,
#simultaneously selecting the best model using the Bayesian Information Criterion
(BIC).

#apply Mclust to the dataset
mclust_wine.new <- Mclust(wine.new.1)

#The output will identify the optimal model in terms of K (number of components)
#and the covariance structure (VEE, EII, VVV and so on).

#summary
summary(mclust_wine.new, parameters = T)
```

```
## ----------------------------------------------------
## Gaussian finite mixture model fitted by EM algorithm
## ----------------------------------------------------
##
## Mclust VVE (ellipsoidal, equal orientation) model with 3 components:
##
##  log-likelihood   n df       BIC       ICL
##        -1151.95 178 42 -2521.535 -2533.047
##
## Clustering table:
##  1  2  3
## 57 75 46
##
## Mixing probabilities:
##         1         2         3
## 0.3183347 0.4251363 0.2565290
##
## Means:
##                         [,1]      [,2]      [,3]
## Alcohol            13.777216 12.317315 13.169326
## Alcalinity of ash  16.862587 20.250183 21.509886
## Total phenols       2.862836  2.237285  1.686443
## Flavanoids          3.015375  2.054074  0.764473
## Color intensity     5.608166  3.149457  7.538594
##
```

```
## Variances:
## [,,1]
##                       Alcohol Alcalinity of ash Total phenols  Flavanoids
## Alcohol            0.196958437       -0.04718413    0.01260051 0.006664289
## Alcalinity of ash -0.047184134        5.84294773    0.09758048 0.209923994
## Total phenols      0.012600513        0.09758048    0.09621475 0.089147018
## Flavanoids         0.006664289        0.20992399    0.08914702 0.145667124
## Color intensity    0.161683387       -0.09391576    0.06008048 0.116991932
##                   Color intensity
## Alcohol                0.16168339
## Alcalinity of ash     -0.09391576
## Total phenols          0.06008048
## Flavanoids             0.11699193
## Color intensity        1.49068510


## [,,2]
##                       Alcohol Alcalinity of ash Total phenols  Flavanoids
## Alcohol            0.26381285       -0.08790694  -0.013665527 -0.03152081
## Alcalinity of ash -0.08790694       10.53174743   0.167511022  0.36659619
## Total phenols     -0.01366553        0.16751102   0.286526877  0.26934731
## Flavanoids        -0.03152081        0.36659619   0.269347308  0.42020618
## Color intensity    0.08427262       -0.19950622   0.005408802  0.02885456
##                   Color intensity
## Alcohol               0.084272617
## Alcalinity of ash    -0.199506222
## Total phenols         0.005408802
## Flavanoids            0.028854560
## Color intensity       0.901471982


## [,,3]
##                       Alcohol Alcalinity of ash Total phenols Flavanoids
## Alcohol            0.32762279       -0.03178398    0.04716889 0.06022526
## Alcalinity of ash -0.03178398        4.92524112    0.08808906 0.18518429
## Total phenols      0.04716889        0.08808906    0.09883688 0.02424513
## Flavanoids         0.06022526        0.18518429    0.02424513 0.13021559
## Color intensity    0.58133861       -0.01368625    0.24319253 0.44657327
##                   Color intensity
## Alcohol                0.58133861
## Alcalinity of ash     -0.01368625
## Total phenols          0.24319253
## Flavanoids             0.44657327
## Color intensity        5.01747916
```

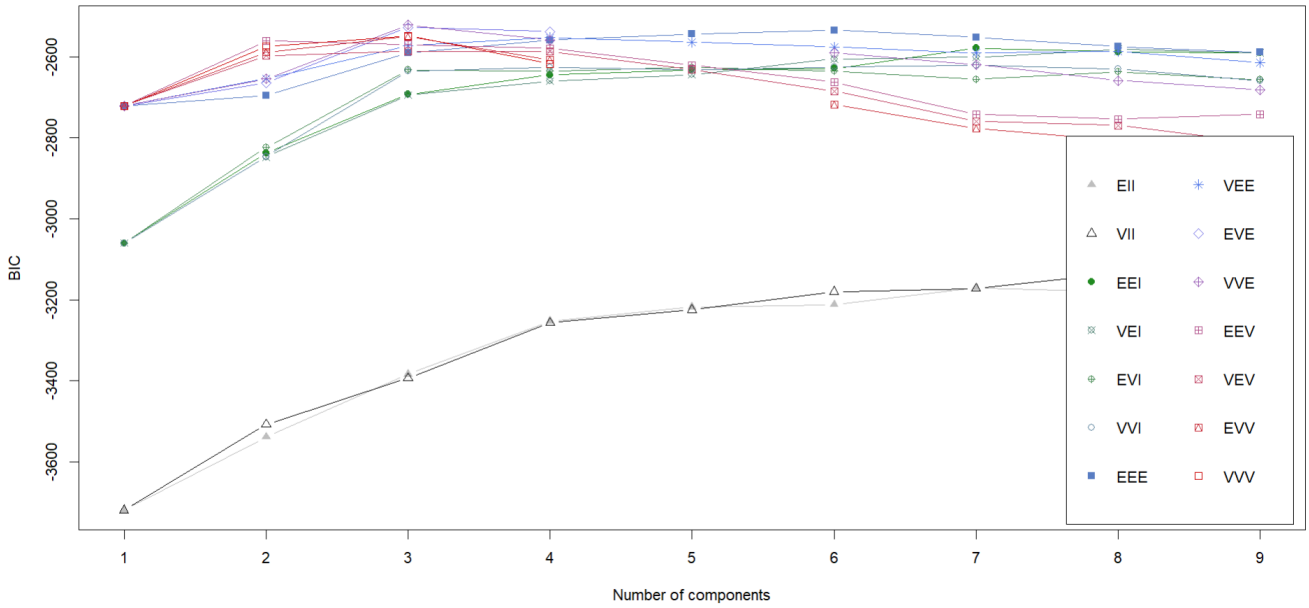
In particular, the output will also give us information about the different structure of the covariance matrix, and, in our case (multivariate), it is a VVE model (Variable Volume, Variable Shape, Equal Orientation).K=3 means that the clustering divides the 178 wines into three main groups, suggesting that there is a difference between the three true classes.

```r
#plots
par(mfrow=c(1,1))

#BIC
mclust_wine.new$BIC
```

```
## Bayesian Information Criterion (BIC):
##         EII       VII       EEI       VEI       EVI       VVI       EEE
## 1 -3719.833 -3719.833 -3059.670 -3059.670 -3059.670 -3059.670 -2720.844
## 2 -3538.932 -3507.842 -2837.600 -2847.023 -2823.600 -2846.417 -2695.768
## 3 -3384.046 -3393.105 -2692.802 -2694.599 -2632.321 -2635.458 -2590.374
## 4 -3252.720 -3256.837 -2644.548 -2659.962 -2635.228 -2626.328 -2558.577
## 5 -3217.757 -3224.870 -2632.131 -2644.227 -2625.496 -2632.261 -2543.229
## 6 -3211.708 -3180.089 -2628.782 -2605.592 -2635.515 -2625.086 -2533.859
## 7 -3170.191 -3171.750 -2578.128 -2600.786 -2655.874 -2620.250 -2551.470
## 8 -3181.041 -3134.527 -2587.991 -2581.414 -2637.217 -2630.295 -2574.661
## 9 -3169.070 -3085.313 -2589.433 -2590.205 -2656.534 -2659.594 -2588.393
##         VEE       EVE       VVE       EEV       VEV       EVV       VVV
## 1 -2720.844 -2720.844 -2720.844 -2720.844 -2720.844 -2720.844 -2720.844
## 2 -2653.906 -2664.530 -2654.751 -2560.806 -2597.430 -2588.813 -2574.362
## 3 -2571.977 -2527.001 -2521.535 -2569.609 -2585.848 -2549.864 -2547.887
## 4 -2552.712 -2537.340 -2558.374 -2578.983 -2586.349 -2608.013 -2617.546
## 5 -2563.604        NA        NA -2620.785 -2631.691        NA        NA
## 6 -2575.397        NA -2589.242 -2661.975 -2684.471 -2717.931        NA
## 7 -2590.068        NA -2619.346 -2742.272 -2759.013 -2777.420        NA
## 8 -2586.452        NA -2658.524 -2753.824 -2769.459 -2806.278        NA
## 9 -2614.940        NA -2681.259 -2741.791 -2812.944        NA -2901.788
##
## Top 3 models based on the BIC criterion:
##     VVE,3     EVE,3     EEE,6
## -2521.535 -2527.001 -2533.859
```

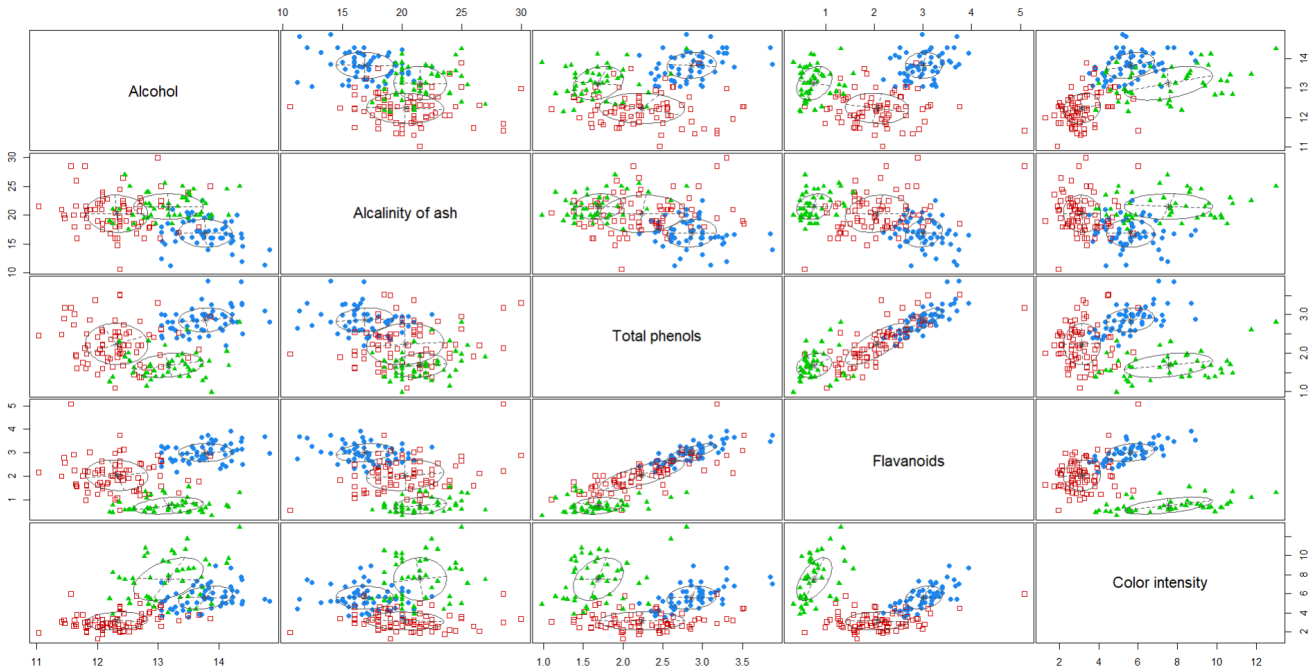
```r
plot(mclust_wine.new, what = "BIC")
```



From this plot, we can understand the shape of the covariance matrix, which gives us information on the choice of the best model (EII, VII, VVV, VEV, VVE, and so on)

The plot shows the BIC (Bayesian Information Criterion) values for different models and K components (clusters). The best selected model will have the lowest BIC value. Since all the BIC values are negative in this case, the lowest value will correspond to the highest point on the plot.The VVE model was selected by Mclust because, numerically, it achieved the lowest BIC score among all the models tested.Specifically, the maximum BIC point occurs at K=3, which was achieved using the VVE model.

```
#classification
plot(mclust_wine.new, what = "classification")
```



The colors and symbols (Red, Green, Blue) no longer represent the true original classes (Class), but the clusters estimates by Mclust. Naturally, each point is assigned to the cluster with the highest posterior probability.

The black ellipses represent the Gaussian components of the mixture model (the clusters) as they were estimated. We can observe that the estimated clusters are very well separated in different feature spaces. In variable pairs such as (Flavanoids, Total phenols), (Color intensity, Flavanoids), and (Color intensity, Total phenols), the three clusters (Red, Green, Blue) form well-distinguished groups with very little overlap. This indicates that the Mclust algorithm has successfully identified the natural groupings, consistent with the original idea that there are three types of wine. The shapes of the black ellipses, in some pairs such as (Total phenols, Flavanoids), appear more elongated and different from the other ellipses. This justifies the choice of the VVE model, which is highly flexible and can adapt well to varying correlations between two variables. In the pair (Alcohol, Alcalinity of ash), we notice that some clusters overlap, suggesting that, in this case, better separation might be achieved by considering combinations of other variable pairs.

```
#uncertainty

#posterior probabilities
mclust_wine.new$z
```

```
##                [,1]         [,2]         [,3]
##   [1,] 9.999925e-01 7.485346e-06 3.460566e-26
##   [2,] 9.808448e-01 1.915525e-02 1.254000e-27
##   [3,] 9.964422e-01 3.557802e-03 5.246755e-25
##   [4,] 9.999998e-01 2.290711e-07 2.652998e-33
##   [5,] 6.168959e-01 3.831041e-01 1.086970e-17
##   [6,] 9.999997e-01 2.992229e-07 1.169333e-32
```

```
##   [7,] 9.999533e-01 4.667531e-05 4.834561e-19
##   [8,] 9.991906e-01 8.094356e-04 9.440815e-17
##   [9,] 9.999982e-01 1.835638e-06 2.168148e-27
##  [10,] 9.999993e-01 6.988487e-07 3.159891e-25
##  [11,] 9.999873e-01 1.265788e-05 4.482834e-28
##  [12,] 9.984097e-01 1.590306e-03 2.780074e-14
##  [13,] 9.997763e-01 2.237224e-04 1.638007e-20
##  [14,] 9.999977e-01 2.318246e-06 4.191323e-42
##  [15,] 1.000000e+00 1.021945e-08 1.017734e-38
##  [16,] 9.999926e-01 7.366219e-06 1.460197e-19
##  [17,] 9.999925e-01 7.463854e-06 9.149351e-22
##  [18,] 9.999875e-01 1.253589e-05 1.881450e-25
##  [19,] 1.000000e+00 4.447742e-10 8.878513e-35
##  [20,] 9.995755e-01 4.245013e-04 3.014929e-26
##  [21,] 9.999876e-01 1.241019e-05 9.630010e-29
##  [22,] 3.311042e-01 6.688958e-01 5.585591e-14
##  [23,] 9.948592e-01 5.140814e-03 4.146001e-24
##  [24,] 1.686359e-01 8.313641e-01 1.414059e-15
##  [25,] 8.177533e-01 1.822467e-01 3.125173e-17
##  [26,] 8.294312e-03 9.917057e-01 9.147520e-16
##  [27,] 9.965271e-01 3.472919e-03 1.279459e-25
##  [28,] 5.879506e-01 4.120494e-01 1.512491e-13
##  [29,] 9.980241e-01 1.975945e-03 6.515363e-24
##  [30,] 9.970793e-01 2.920726e-03 1.141562e-16
##  [31,] 9.987942e-01 1.205824e-03 5.184857e-23
##  [32,] 9.999779e-01 2.214020e-05 3.128873e-22
##  [33,] 9.842949e-01 1.570507e-02 1.078624e-19
##  [34,] 9.970419e-01 2.958069e-03 5.554276e-19
##  [35,] 9.101813e-01 8.981872e-02 3.640689e-15
##  [36,] 9.911053e-01 8.894703e-03 3.278904e-20
##  [37,] 9.835906e-01 1.640940e-02 2.603952e-21
##  [38,] 5.456126e-01 4.543874e-01 2.810472e-15
##  [39,] 7.901915e-01 2.098085e-01 1.426803e-20
##  [40,] 9.999820e-01 1.798455e-05 5.946969e-31
##  [41,] 9.999465e-01 5.347300e-05 2.489666e-30
##  [42,] 9.401453e-01 5.985473e-02 1.363439e-17
##  [43,] 9.999676e-01 3.235296e-05 6.044459e-38
##  [44,] 9.307316e-01 6.926841e-02 3.221374e-19
##  [45,] 9.752109e-01 2.478911e-02 7.319226e-26
##  [46,] 9.996000e-01 3.999606e-04 2.736446e-18
##  [47,] 9.999746e-01 2.537413e-05 9.747530e-32
##  [48,] 9.999932e-01 6.754508e-06 9.027023e-32
##  [49,] 9.999818e-01 1.821204e-05 8.120521e-20
##  [50,] 1.000000e+00 2.124153e-09 2.833513e-24
##  [51,] 9.999117e-01 8.833765e-05 3.117714e-28
##  [52,] 9.996964e-01 3.035648e-04 7.761775e-21
##  [53,] 9.999912e-01 8.802837e-06 2.000033e-43
##  [54,] 9.998849e-01 1.151095e-04 8.929271e-21
##  [55,] 9.998864e-01 1.136337e-04 2.110210e-21
##  [56,] 9.973312e-01 2.668773e-03 5.994988e-17
##  [57,] 9.999958e-01 4.233733e-06 9.040180e-26
```

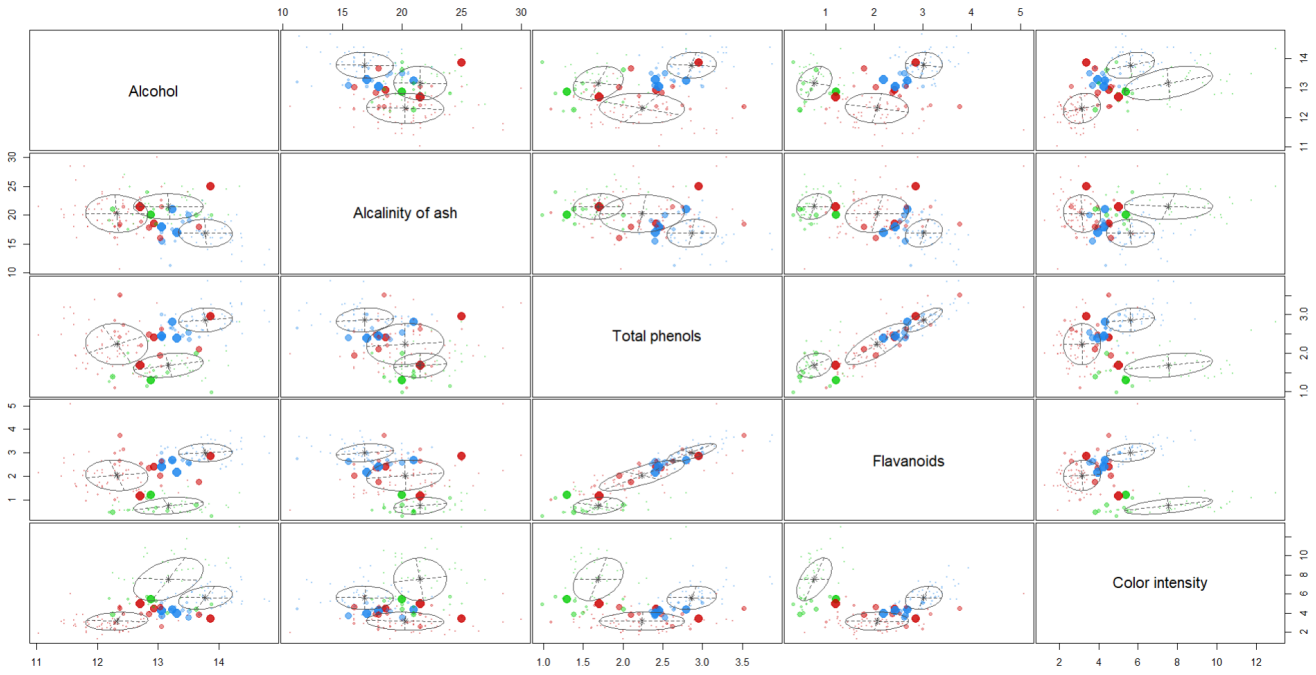
```
##  [58,] 9.996157e-01 3.843416e-04 7.985078e-28
##  [59,] 9.999966e-01 3.365662e-06 4.274328e-36
##  [60,] 5.610621e-10 9.999988e-01 1.219130e-06
##  [61,] 6.424272e-08 9.999837e-01 1.628286e-05
##  [62,] 1.149305e-04 9.968780e-01 3.007049e-03
##  [63,] 1.918298e-01 8.081701e-01 1.213379e-07
##  [64,] 3.460597e-03 9.965394e-01 1.261556e-30
##  [65,] 9.476217e-06 9.999905e-01 1.138883e-08
##  [66,] 5.180798e-02 9.481920e-01 1.996191e-17
##  [67,] 9.964050e-01 3.594970e-03 3.826233e-30
##  [68,] 4.500935e-04 9.995499e-01 1.571893e-08
##  [69,] 8.901580e-06 9.999902e-01 9.148919e-07
##  [70,] 6.259762e-07 9.999971e-01 2.264377e-06
##  [71,] 7.006670e-10 9.772440e-01 2.275604e-02
##  [72,] 3.925659e-01 6.074341e-01 6.382085e-19
##  [73,] 4.496402e-04 9.995093e-01 4.110198e-05
##  [74,] 1.402801e-06 9.999986e-01 4.647909e-21
##  [75,] 1.805221e-10 1.000000e+00 2.133561e-18
##  [76,] 2.211981e-07 9.999997e-01 8.396565e-08
##  [77,] 1.624403e-01 8.375597e-01 7.733365e-10
##  [78,] 3.774139e-08 9.999963e-01 3.699330e-06
##  [79,] 5.765920e-04 9.994234e-01 1.426734e-11
##  [80,] 9.023781e-04 9.990976e-01 1.556720e-18
##  [81,] 9.081292e-05 9.999092e-01 1.811428e-16
##  [82,] 5.812696e-02 9.418730e-01 3.479063e-15
##  [83,] 9.411216e-09 9.999982e-01 1.794440e-06
##  [84,] 3.273223e-05 9.844961e-01 1.547115e-02
##  [85,] 4.688105e-05 9.999531e-01 5.355689e-15
##  [86,] 2.070514e-03 9.979295e-01 7.798287e-12
##  [87,] 1.606877e-07 9.999989e-01 9.571867e-07
##  [88,] 4.532941e-11 9.999964e-01 3.557711e-06
##  [89,] 1.399079e-08 9.999999e-01 7.061946e-08
##  [90,] 1.026613e-08 9.999998e-01 1.561151e-07
##  [91,] 5.989768e-07 9.999985e-01 8.774497e-07
##  [92,] 9.104817e-10 9.937169e-01 6.283063e-03
##  [93,] 9.469812e-07 9.996824e-01 3.166120e-04
##  [94,] 1.071746e-03 9.989283e-01 3.235593e-17
##  [95,] 1.415982e-07 9.999999e-01 2.530936e-20
##  [96,] 2.098039e-03 9.979020e-01 3.141246e-16
##  [97,] 2.737966e-11 9.913288e-01 8.671223e-03
##  [98,] 1.042487e-02 9.895751e-01 2.377052e-21
##  [99,] 9.724683e-02 9.027532e-01 2.071571e-41
## [100,] 1.323460e-02 9.867654e-01 6.169793e-29
## [101,] 3.114978e-04 9.996885e-01 1.903926e-14
## [102,] 2.818257e-06 9.999556e-01 4.162162e-05
## [103,] 3.966092e-05 9.999603e-01 1.439330e-13
## [104,] 1.667615e-08 1.000000e+00 5.984692e-11
## [105,] 1.109732e-04 9.998890e-01 4.640227e-10
## [106,] 3.120937e-06 9.999966e-01 2.491438e-07
## [107,] 2.304648e-05 9.999770e-01 1.988092e-09
## [108,] 8.851423e-07 9.999571e-01 4.198834e-05
```

```
## [109,] 9.550610e-05 9.999045e-01 2.304404e-13
## [110,] 3.845756e-05 9.999615e-01 3.737168e-26
## [111,] 9.333699e-08 9.999999e-01 3.491630e-25
## [112,] 5.457339e-04 9.994543e-01 7.578483e-16
## [113,] 9.525346e-07 9.999990e-01 2.620977e-09
## [114,] 1.587967e-08 1.000000e+00 4.844509e-13
## [115,] 9.113751e-06 9.999909e-01 7.394352e-15
## [116,] 2.879702e-09 1.000000e+00 3.920591e-16
## [117,] 6.148159e-08 9.999999e-01 4.399392e-08
## [118,] 2.689347e-05 9.999731e-01 8.956362e-11
## [119,] 2.855097e-05 9.999384e-01 3.300815e-05
## [120,] 2.048384e-06 9.999980e-01 1.563846e-09
## [121,] 4.395417e-06 9.999956e-01 6.108260e-25
## [122,] 3.091510e-09 1.000000e+00 3.993365e-49
## [123,] 5.742523e-07 9.999994e-01 1.139152e-10
## [124,] 7.935201e-02 9.206480e-01 1.105267e-18
## [125,] 1.952990e-04 9.998047e-01 4.286937e-27
## [126,] 2.839723e-04 9.997160e-01 3.649668e-20
## [127,] 2.619836e-02 9.738016e-01 2.813085e-24
## [128,] 1.182986e-09 1.000000e+00 8.597239e-11
## [129,] 2.519757e-05 9.999748e-01 2.384124e-14
## [130,] 2.791801e-07 9.999997e-01 1.848553e-08
## [131,] 1.168039e-05 9.967739e-01 3.214383e-03
## [132,] 6.453102e-07 3.723720e-01 6.276273e-01
## [133,] 5.021310e-11 2.449703e-02 9.755030e-01
## [134,] 1.148864e-07 5.886725e-01 4.113273e-01
## [135,] 1.202929e-13 1.807046e-02 9.819295e-01
## [136,] 5.766241e-14 2.918618e-05 9.999708e-01
## [137,] 3.339510e-14 1.265865e-01 8.734135e-01
## [138,] 2.910931e-17 3.179676e-03 9.968203e-01
## [139,] 1.006062e-12 4.784214e-04 9.995216e-01
## [140,] 5.950062e-19 1.184119e-03 9.988159e-01
## [141,] 2.756883e-13 1.350819e-02 9.864918e-01
## [142,] 2.878393e-12 1.470418e-03 9.985296e-01
## [143,] 2.683415e-13 8.696797e-03 9.913032e-01
## [144,] 5.283902e-09 9.245574e-02 9.075443e-01
## [145,] 6.391163e-15 6.673195e-07 9.999993e-01
## [146,] 1.305309e-11 5.599798e-02 9.440020e-01
## [147,] 3.201890e-11 3.350119e-02 9.664988e-01
## [148,] 1.074603e-16 2.210294e-07 9.999998e-01
## [149,] 1.099571e-16 1.652179e-09 1.000000e+00
## [150,] 9.556114e-12 8.031678e-10 1.000000e+00
## [151,] 7.646586e-10 1.015584e-07 9.999999e-01
## [152,] 8.539297e-16 2.536189e-14 1.000000e+00
## [153,] 3.630224e-12 6.481774e-05 9.999352e-01
## [154,] 1.783325e-17 1.843821e-14 1.000000e+00
## [155,] 2.879757e-16 1.187176e-06 9.999988e-01
## [156,] 4.190185e-17 2.909674e-08 1.000000e+00
## [157,] 8.750792e-14 1.380865e-10 1.000000e+00
## [158,] 2.732036e-23 1.848720e-07 9.999998e-01
## [159,] 6.060849e-26 3.512141e-26 1.000000e+00
```

```
## [160,] 3.792319e-24 2.808743e-20 1.000000e+00
## [161,] 3.180667e-17 1.969432e-06 9.999980e-01
## [162,] 6.848217e-13 1.318313e-04 9.998682e-01
## [163,] 1.665550e-14 8.929626e-04 9.991070e-01
## [164,] 8.474883e-10 3.984349e-02 9.601565e-01
## [165,] 1.560094e-16 9.041830e-12 1.000000e+00
## [166,] 3.551630e-14 3.276766e-05 9.999672e-01
## [167,] 5.067884e-19 1.205597e-15 1.000000e+00
## [168,] 2.853093e-19 2.376696e-13 1.000000e+00
## [169,] 4.685294e-16 1.168767e-09 1.000000e+00
## [170,] 7.448225e-17 1.234505e-09 1.000000e+00
## [171,] 4.408174e-14 1.444468e-02 9.855553e-01
## [172,] 8.215755e-20 3.504980e-12 1.000000e+00
## [173,] 9.900256e-16 7.185340e-13 1.000000e+00
## [174,] 1.726489e-14 8.604469e-08 9.999999e-01
## [175,] 3.610403e-15 5.307190e-07 9.999995e-01
## [176,] 2.133020e-18 1.053616e-13 1.000000e+00
## [177,] 3.072263e-17 3.514186e-11 1.000000e+00
## [178,] 3.449346e-19 1.446146e-12 1.000000e+00
```

This command measures the posterior probability that observation (i) belongs to cluster (k) (the probabilities are calculated from the estimated mixture model). It is important because the uncertainty is computed as (1 - *k* *z*{ik}). The higher the calculated posterior probability, the lower the uncertainty. Mclust calculates the posterior probabilities for each observation and assigns them to clusters based on these values. However, there can still be issues, as clusters may overlap or two or more components may have similar posterior probabilities, which increases uncertainty.

```
plot(mclust_wine.new, what = "uncertainty")
```



This is a way to indicate how confident we are that each point belongs to its assigned cluster, and where uncertainty manifests itself in the variable space.

```
#values for uncertainty
uncertainty <- 1 - apply(mclust_wine.new$z, 1, max)
mclust_wine.new$uncertainty
```

```
##   [1] 7.485346e-06 1.915525e-02 3.557802e-03 2.290711e-07 3.831041e-01
##   [6] 2.992229e-07 4.667531e-05 8.094356e-04 1.835638e-06 6.988487e-07
##  [11] 1.265788e-05 1.590306e-03 2.237224e-04 2.318246e-06 1.021945e-08
##  [16] 7.366219e-06 7.463854e-06 1.253589e-05 4.447742e-10 4.245013e-04
##  [21] 1.241019e-05 3.311042e-01 5.140814e-03 1.686359e-01 1.822467e-01
##  [26] 8.294312e-03 3.472919e-03 4.120494e-01 1.975945e-03 2.920726e-03
##  [31] 1.205824e-03 2.214020e-05 1.570507e-02 2.958069e-03 8.981872e-02
##  [36] 8.894703e-03 1.640940e-02 4.543874e-01 2.098085e-01 1.798455e-05
##  [41] 5.347300e-05 5.985473e-02 3.235296e-05 6.926841e-02 2.478911e-02
##  [46] 3.999606e-04 2.537413e-05 6.754508e-06 1.821204e-05 2.124153e-09
##  [51] 8.833765e-05 3.035648e-04 8.802837e-06 1.151095e-04 1.136337e-04
##  [56] 2.668773e-03 4.233733e-06 3.843416e-04 3.365662e-06 1.219691e-06
##  [61] 1.634710e-05 3.121980e-03 1.918299e-01 3.460597e-03 9.487606e-06
##  [66] 5.180798e-02 3.594970e-03 4.501093e-04 9.816472e-06 2.890353e-06
##  [71] 2.275604e-02 3.925659e-01 4.907421e-04 1.402801e-06 1.805220e-10
##  [76] 3.051637e-07 1.624403e-01 3.737071e-06 5.765920e-04 9.023781e-04
##  [81] 9.081292e-05 5.812696e-02 1.803852e-06 1.550389e-02 4.688105e-05
##  [86] 2.070514e-03 1.117874e-06 3.557756e-06 8.461026e-08 1.663812e-07
##  [91] 1.476427e-06 6.283064e-03 3.175589e-04 1.071746e-03 1.415982e-07
##  [96] 2.098039e-03 8.671223e-03 1.042487e-02 9.724683e-02 1.323460e-02
## [101] 3.114978e-04 4.443988e-05 3.966092e-05 1.673600e-08 1.109737e-04
## [106] 3.370081e-06 2.304846e-05 4.287348e-05 9.550610e-05 3.845756e-05
## [111] 9.333699e-08 5.457339e-04 9.551556e-07 1.588016e-08 9.113751e-06
## [116] 2.879702e-09 1.054755e-07 2.689356e-05 6.155912e-05 2.049948e-06
## [121] 4.395417e-06 3.091509e-09 5.743662e-07 7.935201e-02 1.952990e-04
## [126] 2.839723e-04 2.619836e-02 1.268958e-09 2.519757e-05 2.976657e-07
## [131] 3.226063e-03 3.723727e-01 2.449703e-02 4.113275e-01 1.807046e-02
## [136] 2.918618e-05 1.265865e-01 3.179676e-03 4.784214e-04 1.184119e-03
## [141] 1.350819e-02 1.470418e-03 8.696797e-03 9.245575e-02 6.673195e-07
## [146] 5.599798e-02 3.350119e-02 2.210294e-07 1.652180e-09 8.127239e-10
## [151] 1.023230e-07 2.620126e-14 6.481774e-05 1.842970e-14 1.187176e-06
## [156] 2.909674e-08 1.381739e-10 1.848720e-07 0.000000e+00 0.000000e+00
## [161] 1.969432e-06 1.318313e-04 8.929626e-04 3.984350e-02 9.041878e-12
## [166] 3.276766e-05 1.110223e-15 2.375877e-13 1.168768e-09 1.234505e-09
## [171] 1.444468e-02 3.504974e-12 7.194245e-13 8.604470e-08 5.307190e-07
## [176] 1.052491e-13 3.514189e-11 1.446177e-12
```

By comparing the values of the posterior probabilities and the uncertainty, we can observe that as the posterior probability increases, the uncertainty decreases.

(c) Compare the estimated clustering based on the posterior probabilities with the true one and compute the number of misclassified units.

```r
#contingency table
contingency_table <- table(True_class = wine.new$Class, Estimated_cluster =
mclust_wine.new$classification); contingency_table
```

```
##           Estimated_cluster
## True_class  1  2  3
##          1 56  3  0
##          2  1 70  0
##          3  0  2 46
```

```r
#missclassified units
misclassified_units <- sum(contingency_table) - sum(diag(contingency_table));
misclassified_units
```

```
## [1] 6
```

The number of misclassified units is 6. This result is obtained very quickly, as it is sufficient to refer to the off-diagonal elements of the contingency table. The elements on the main diagonal are correctly classified units, while each off-diagonal cell represents a wine that was misassigned by the Mclust algorithm to a cluster different from its true origin. To understand whether Mclust has performed well or poorly, it is not enough to look at the number of misclassified units; it is also necessary to calculate the error rate.

```r
mc_classification_error <- misclassified_units / (mclust_wine.new$n);
mc_classification_error
```

```
## [1] 0.03370787
```

```r
mc_classification_error1 <- 6 / 178; mc_classification_error1
```

```
## [1] 0.03370787
```

```r
#accuracy rate: 1 - error rate
accurancy_rate <- 1 - mc_classification_error; accurancy_rate
```

```
## [1] 0.9662921
```

Such a high accuracy rate shows that the Mclust function we used to discover groupings in the data—without prior knowledge of the true groupings—performed correctly. This excellent result is mainly due to the choice of response variables, which proved to be highly discriminative. We can conclude that the result is excellent and that Mclust successfully identified the underlying clustering structure of the wines.

(d) Use function mix.norm.R to estimate model parameters by fixing the number of components to k = 3. Compare the estimation results, in terms of mean and component weights, with those obtained with function Mclust.

```r
mix_norm <- function(Y,k,start){
  n = dim(Y)[1]
  r = dim(Y)[2]
```

```
  Si = cov(Y)

  if(start == 0){
    mu = matrix(0,r,k)
    for(u in 1:k){
      for(j in 1:r){
        mu[j,u] = quantile(Y[,j], 1*u/(k+1))
        piv = rep(1, k)/k
      }
    }
  }else{
    mu = t(rmvnorm(k, colMeans(Y), cov(Y)))
    piv = runif(k)
    piv = piv / sum(piv)
  }
  #log-likelihood
  Pc = matrix(0,n,k)
  for(u in 1:k){
    Pc[,u] = dmvnorm(Y, mu[,u], Si)
  }
  Pj = Pc %*% diag(piv)
  pm = rowSums(Pj)
  lk = sum(log(pm))
  #iterate until convergence
  lko = lk; it = 0
  while((abs(lk-lko) / abs(lko) > 10^-10 & it < 10^5) | it == 0){
    lko = lk
    it = it +1
    #---- E-step ----
    Zu = diag(1/pm) %*% Pj
    #---- M-step ----
    piv = colSums(Zu) / n
    #Update mu and sigma
    mu = (t(Y)%*%Zu) %*% diag(1/colSums(Zu))
    Si = matrix(0,r,r)
    for(u in 1:k){
      Si = Si + t(Y-rep(1,n) %*% t(mu[,u])) %*% diag(Zu[,u]) %*% as.matrix(Y-
rep(1,n) %*% t(mu[,u]))
    }
    Si = Si/n
    Si = (Si + t(Si)) / 2
    #Compute likelihood
    Pc = matrix(0,n,k)
    for(u in 1:k){
      Pc[,u] = dmvnorm(Y, mu[,u], Si)
    }
    Pj = Pc %*% diag(piv)
    pm = rowSums(Pj)
    lk = sum(log(pm))

    if(it / 1 == floor(it/1)){
```

```
      print(c(it,lk, lk-lko))
    }
  }
  #Final arguments
  Pp = ((1/pm) %o% piv) * Pc
  np = (k-1)+(k*r) + r*(r*1)/2
  aic = -2*lk + 2*np
  bic = -2*lk + np*log(n)

  out = list(piv=piv, mu=mu, Si=Si, lk=lk, np=np, aic=aic, bic=bic, Pp=Pp)
}
```

A custom function mix.norm.R, which implements the Expectation-Maximization algorithm to estimate the parameters of a Gaussian Mixture Model (GMM) under the assumption of equal and unconstrained covariance matrices.

```
wine.new.1.matrix <- as.matrix(wine.new.1)
```

We need to convert the results into a matrix because the for loop implemented above requires i and j elements. Therefore, We have to convert my dataset—excluding the true classes—into a matrix in order to use the mix.norm.R function.

```
#estimate the model with the function mix.norm.R
mix_norm_results <- mix_norm(Y = wine.new.1.matrix, k = 3, start = 0)
```

```
## [1]     1.00000 -1309.79790    50.56044
## [1]     2.0000000 -1308.9785195     0.8193844
## [1]     3.0000000 -1308.4290383     0.5494812
## [1]     4.0000000 -1307.9512409     0.4777974
## [1]     5.0000000 -1307.4346258     0.5166151
## [1]     6.0000000 -1306.7771540     0.6574718
## [1]     7.0000000 -1305.8439693     0.9331847
## [1]     8.000000 -1304.431099     1.412871
## [1]     9.000000 -1302.237627     2.193471
## [1]    10.000000 -1298.894316     3.343311
## [1]    11.000000 -1294.143782     4.750535
## [1]    12.000000 -1288.194727     5.949054
## [1]    13.000000 -1281.967094     6.227633
## [1]    14.000000 -1276.647456     5.319639
## [1]    15.000000 -1272.260309     4.387146
## [1]    16.000000 -1267.140512     5.119798
## [1]    17.000000 -1259.516709     7.623802
## [1]    18.00000 -1247.25645    12.26026
## [1]    19.00000 -1230.34878    16.90767
## [1]    20.00000 -1216.94682    13.40195
## [1]    21.000000 -1212.877712     4.069112
## [1]    22.0000000 -1212.3767753     0.5009368
## [1]  2.300000e+01 -1.212296e+03  8.117622e-02
## [1]  2.400000e+01 -1.212280e+03  1.602185e-02
## [1]  2.500000e+01 -1.212275e+03  4.215395e-03
## [1]  2.600000e+01 -1.212274e+03  1.499354e-03
```

```
## [1]  2.700000e+01 -1.212273e+03  6.441695e-04
## [1]  2.800000e+01 -1.212273e+03  3.020005e-04
## [1]  2.900000e+01 -1.212273e+03  1.468007e-04
## [1]  3.000000e+01 -1.212273e+03  7.243734e-05
## [1]  3.100000e+01 -1.212273e+03  3.597670e-05
## [1]  3.200000e+01 -1.212273e+03  1.792194e-05
## [1]  3.300000e+01 -1.212273e+03  8.941225e-06
## [1]  3.400000e+01 -1.212273e+03  4.464329e-06
## [1]  3.500000e+01 -1.212273e+03  2.230053e-06
## [1]  3.600000e+01 -1.212273e+03  1.114286e-06
## [1]  3.700000e+01 -1.212273e+03  5.568734e-07
## [1]  3.800000e+01 -1.212273e+03  2.783354e-07
## [1]  3.900000e+01 -1.212273e+03  1.391284e-07
## [1]  4.000000e+01 -1.212273e+03  6.954861e-08
```

```
#component weights EM algorithm
mix_norm_results$piv
```

```
## [1] 0.3877163 0.2574567 0.3548270
```

```
#component weights Mclust
mclust_wine.new$parameters$pro
```

```
## [1] 0.3183347 0.4251363 0.2565290
```

```
#table for the components weights
table_weights <- data.frame(
  EM_algorithm_weights = mix_norm_results$piv,
  Mclust_weights = mclust_wine.new$parameters$pro
); table_weights
```

```
##   EM_algorithm_weights Mclust_weights
## 1            0.3877163      0.3183347
## 2            0.2574567      0.4251363
## 3            0.3548270      0.2565290
```

They are different because the EM algorithm uses a different starting point and likely converges to a different local maximum.

```
#component means EM
mix_norm_results$mu
```

```
##                        [,1]       [,2]      [,3]
## Alcohol           12.296023 13.1672787 13.649597
## Alcalinity of ash 20.134811 21.5025368 17.339090
## Total phenols      2.170446  1.6891458  2.871014
## Flavanoids         1.942499  0.7627339  3.043061
## Color intensity    3.070661  7.5260360  5.439035
```

```
#component means Mclust
mclust_wine.new$parameters$mean
```

```
##                        [,1]      [,2]      [,3]
## Alcohol           13.777216 12.317315 13.169326
## Alcalinity of ash 16.862587 20.250183 21.509886
## Total phenols      2.862836  2.237285  1.686443
## Flavanoids         3.015375  2.054074  0.764473
## Color intensity    5.608166  3.149457  7.538594
```

```
#table for the means
table_means<- data.frame(
  EM_means = mix_norm_results$mu,
  Mclust_means = mclust_wine.new$parameters$mean
); table_means
```

```
##                   EM_means.1 EM_means.2 EM_means.3 Mclust_means.1
## Alcohol            12.296023 13.1672787  13.649597      13.777216
## Alcalinity of ash  20.134811 21.5025368  17.339090      16.862587
## Total phenols       2.170446  1.6891458   2.871014       2.862836
## Flavanoids          1.942499  0.7627339   3.043061       3.015375
## Color intensity     3.070661  7.5260360   5.439035       5.608166
```

```
##                   Mclust_means.2 Mclust_means.3
## Alcohol                12.317315      13.169326
## Alcalinity of ash      20.250183      21.509886
## Total phenols           2.237285       1.686443
## Flavanoids              2.054074       0.764473
## Color intensity         3.149457       7.538594
```

```
#posterior probabilities Mclust
round(head(mclust_wine.new$z),3)
```

```
##       [,1]  [,2] [,3]
## [1,] 1.000 0.000    0
## [2,] 0.981 0.019    0
## [3,] 0.996 0.004    0
## [4,] 1.000 0.000    0
## [5,] 0.617 0.383    0
## [6,] 1.000 0.000    0
```

```
#posterior probabilities EM
round(head(mix_norm_results$Pp),3)
```

```
##       [,1] [,2]  [,3]
## [1,] 0.000    0 1.000
## [2,] 0.003    0 0.997
## [3,] 0.005    0 0.995
## [4,] 0.000    0 1.000
## [5,] 0.220    0 0.780
## [6,] 0.000    0 1.000
```

```
#table for the posterior probabilities
table_post_probs <- data.frame(
  EM_post_probs = round(head(mix_norm_results$Pp),3),
```

```
  Mclust_post_probs = round(head(mclust_wine.new$z),3)
); table_post_probs

##   EM_post_probs.1 EM_post_probs.2 EM_post_probs.3 Mclust_post_probs.1
## 1           0.000               0           1.000                1.000
## 2           0.003               0           0.997                0.981
## 3           0.005               0           0.995                0.996
## 4           0.000               0           1.000                1.000
## 5           0.220               0           0.780                0.617
## 6           0.000               0           1.000                1.000
##   Mclust_post_probs.2 Mclust_post_probs.3
## 1               0.000                   0
## 2               0.019                   0
## 3               0.004                   0
## 4               0.000                   0
## 5               0.383                   0
## 6               0.000                   0
```

By comparing all the results for component weights, means, and posterior probabilities, it is evident that they differ between Mclust and the EM algorithm. This occurs because the EM algorithm uses a different starting point, which leads to convergence to a local maximum different from the one found using Mclust.

To verify whether the models have indeed reached two different maxima, we can analyze the log-likelihoods of both models, since both use the incomplete-data log-likelihood.

```
mix_norm_results$lk

## [1] -1212.273

mclust_wine.new$loglik

## [1] -1151.95
```

Furthermore, the divergence from the results may also be due to the fact that Mclust identified a VEV model (Variable volumes between clusters, Equal shapes across clusters, Variable orientations between clusters), while the EM algorithm, without imposing any constraints, automatically uses the VVV model (all cluster characteristics are variable).

In the presence of multiple local maxima, the EM algorithm can converge to only one of them. It is advisable to use a random initialization strategy if we want to avoid this problem.

(e) Used the maximum a posteriori rule to cluster units in the three components on the basis of the posterior probabilities obtained at convergence of mix.norm.R and collected in Pp. Compare this allocation with the true classification and compute the number of misclassified units.

The posterior probabilities (zˆiu) can be used for clustering, that is, to allocate subjects to the (k) components (clusters). Based on the output of the EM algorithm, we assign subject (i) to component (u) when (zˆiu = z*i), where (z*i) is the maximum among (zˆi1, ..., zˆik) (maximum a

posteriori, MAP, rule). We need to reorder the results because the alignment of the clusters may not correspond to the order of the true classes.

```
posterior_probs <- cbind(mix_norm_results$Pp[,3], mix_norm_results$Pp[,1],
mix_norm_results$Pp[,2])
round(head(mclust_wine.new$z),4)
```

```
##        [,1]   [,2] [,3]
## [1,] 1.0000 0.0000    0
## [2,] 0.9808 0.0192    0
## [3,] 0.9964 0.0036    0
## [4,] 1.0000 0.0000    0
## [5,] 0.6169 0.3831    0
## [6,] 1.0000 0.0000    0
```

```
round(head(posterior_probs),4)
```

```
##        [,1]   [,2] [,3]
## [1,] 1.0000 0.0000    0
## [2,] 0.9969 0.0031    0
## [3,] 0.9950 0.0050    0
## [4,] 1.0000 0.0000    0
## [5,] 0.7797 0.2203    0
## [6,] 1.0000 0.0000    0
```

```
#Cluster assignment (MAP Classification) by finding the maximum of the posterior
probabilities.
mix_norm_classification <- apply(posterior_probs, 1, which.max)

#contingency table
contingency_table2 <- table(True_class = wine.new$Class, Estimated_cluster =
mix_norm_classification); contingency_table2
```

```
##           Estimated_cluster
## True_class  1  2  3
##          1 56  3  0
##          2  8 63  0
##          3  0  2 46
```

```
#missclassified units
misclassified_units2 <- sum(contingency_table2) - sum(diag(contingency_table2));
misclassified_units2
```

```
## [1] 13
```

```
#error rate = misclassified units / total rows of Pp
mc_classification_error2 <- misclassified_units2 / nrow(mix_norm_results$Pp);
mc_classification_error2
```

```
## [1] 0.07303371
```

```
#accuracy rate
accurancy_rate2 <- 1 - mc_classification_error2; accurancy_rate2
```

```
## [1] 0.9269663

#table with misclassification errors
table_errors <- data.frame(
  MAP = mc_classification_error2,
  Mclust = mc_classification_error
); table_errors

##           MAP     Mclust
## 1   0.07303371 0.03370787
```

Both clustering methods, MAP and Mclust, show high accuracy in classifying the units, with misclassification rates of 7.3% and 3.4%, respectively.

The slight superiority of Mclust is due to the constraints on the covariance model and the more robust initialization procedures, but both methods effectively capture the underlying structure of the data.